



University of Deusto

Bachelor in Data Science and Artificial Intelligence

INTEGRATION OF VISION LANGUAGE MODELS (VLMS) IN A
MULTI-AGENT REINFORCEMENT LEARNING (MARL)
ENVIRONMENT FOR COOPERATIVE ROBOTIC TASKS IN NVIDIA
ISAAC SIM

Student Unai Olaizola Osa

DNI 73032586L

Email unai.olaizola.osa@opendeusto.es

GitHub repository [VLMS IsaacSim](#)

April 17, 2026

Acknowledgments

Abstract

This project proposes the development and implementation of a Multi-Agent Reinforcement Learning (MARL) system integrated with Vision Language Models (VLMs) for the execution of cooperative robotic tasks within a simulated NVIDIA Isaac Sim environment.

One of the major challenges lies in translating commands from a natural language format into precise robotic actions. To address this, the reasoning capabilities of the VLMs have been exploited for complex instruction interpretation and decision-making. A controlled environment where robotic agents must cooperate to manipulate and respond to objects in order to achieve a shared goal has been designed and shaped for Reinforcement Learning training using NVIDIA's Isaac Lab framework. The integration of these models has enhanced generalization and promote a more cooperative behavior of the agents against variations in the scene. The learning efficiency has been evaluated in addition to metrics such as the stability of the policies and the transferable capacity of the model(s). Additionally, the project includes an in-depth analysis of state-of-the-art (SOTA) models, providing a comprehensive review of the current methodologies and core concepts that form the theoretical foundation of the proposed architecture, presented in the documentation.

The expected outcome is a demonstration and reflection of the empowerment of multi-modal models within robotic environments, which can be applied to real-world problems.

Index terms

Vision language models, reinforcement learning, multi-agent environment

Contents

Acknowledgments	i
1 Introduction	1
1.1 Problem context	1
1.2 Motivation	1
1.3 Main objectives and specific objectives	1
1.4 Project scope and limitations	1
2 Theoretical foundations	2
2.1 Vision Language Models	2
2.1.1 History of VLMs	2
2.1.2 Elements of a VLMs	2
2.2 Reinforcement Learning	3
2.3 Multi-agent systems	3
2.4 State of the Art	3
2.4.1 VLMs evolution	3
2.4.2 Current models	3
2.4.3 Benchmarks	3
2.4.4 VLMs in robotics	3
2.4.5 Frameworks	3
3 Solution approach	4
3.1 Solution proposal	4
3.2 System architecture	4
3.3 Tools used	4
3.4 Workflow	4

4	Solution development	5
4.1	IsaacSim environment definition	5
4.1.1	Agents	5
4.1.2	Cameras	5
4.1.3	Physics	5
4.2	Individual component operation	5
5	Experimentation and evolution	6
5.1	First scenarios	6
5.2	Last scenario	6
5.3	Successful and failure tries	6
5.4	Results obtained in each scenario	6
5.5	Result discussion	6
6	Conclusions	7
6.1	Conclusions and lessons learned	7
6.2	Limitations found	7
6.3	Future work	8
7	Appendix	10
7.1	Instalation guide	10
7.2	Relevant code	10
7.3	Prompts used	10
7.4	Result table	10
7.5	Tensorboard analysis	10

List of Figures

List of Pseudocodes

1. INTRODUCTION

1.1 PROBLEM CONTEXT

1.2 MOTIVATION

1.3 MAIN OBJECTIVES AND SPECIFIC OBJECTIVES

1.4 PROJECT SCOPE AND LIMITATIONS

2. THEORETICAL FOUNDATIONS

2.1 VISION LANGUAGE MODELS

Vision language models (VLMs) are artificial intelligence (AI) models that combine both computer vision (CV) and natural language processing (NLP) techniques and capabilities, these models are designed to jointly understand and reason visual and textual information simultaneously [1]. Among the most well-known models which will be covered when corresponded, models such as GPT-5, Gemini and LLaVA could be highlighted.

In comparison to the traditional models that process data modalities independently, VLMs have been shaped to align textual and visual data, enabling to reason complex scenes and natural language instructions.

2.1.1 History of VLMs

2.1.2 Elements of a VLMs

Inspired in other previous model architectures, vision language models consisted on two main components: the *language encoder* and the *visual encoder*. Compared to other *seq2seq* architectures like encoder-only, decoder-only or encoder-decoder, VLMs consist on 2 encoders, one for each data modality encoding task.

2.1.2.1 Language encoder

This first encoder captures the semantic relations and meanings between the words in the textual prompt given and turns them into *text embeddings* to be processed, carrying the same encoding process seen in the majority of architectures. Most of VLMs use a transformer-based neural network,

2.2 REINFORCEMENT LEARNING

2.3 MULTI-AGENT SYSTEMS

2.4 STATE OF THE ART

2.4.1 VLMs evolution

2.4.2 Current models

2.4.3 Benchmarks

2.4.4 VLMs in robotics

2.4.5 Frameworks

3. SOLUTION APPROACH

3.1 SOLUTION PROPOSAL

3.2 SYSTEM ARCHITECTURE

3.3 TOOLS USED

3.4 WORKFLOW

4. SOLUTION DEVELOPMENT

4.1 ISAACSIM ENVIRONMENT DEFINITION

4.1.1 Agents

4.1.2 Cameras

4.1.3 Physics

4.2 INDIVIDUAL COMPONENT OPERATION

5. EXPERIMENTATION AND EVOLUTION

5.1 FIRST SCENARIOS

5.2 LAST SCENARIO

5.3 SUCCESSFUL AND FAILURE TRIES

5.4 RESULTS OBTAINED IN EACH SCENARIO

5.5 RESULT DISCUSSION

6. CONCLUSIONS

6.1 CONCLUSIONS AND LESSONS LEARNED

6.2 LIMITATIONS FOUND

The main drawbacks I have found during the development and research process during the whole work have been mainly hardware-related as the specific software requirements necessary to carry on the development are severely demanding due to the fact that the majority of the processes are run and rendered in the GPU. Although I have had access to a good graphical processing unit, the other components of the my working device have fallen behind. Taking for instance the minimum requirements of the used framework *IsaacSim*, I found myself in the limits of the bare minimum, and some of my components such as the CPU were worse than the required. However, I have been able to carry on anyways although there have been a few moments where the I have really felt the bottleneck. When it comes those moments I have felt limited and slowed down due to my equipment are the following:

- I have needed to split some of the working components of the pipeline process inside *IsaacSim* as the whole process would not fit in my GPU's VRAM. The simple fact of opening the framework already consumed close to a quarter of the graphic card's total capacity, and starting the server to listen and interact with the VLM calls alongside left little to no room for more processes in the background. The fact of working on the edge of the minimum hardware requirements also slowed the rendering of visual work and reduced the number of frames per second, nevertheless, this did not obstruct the development process.
- When it comes to making the most of the reinforcement learning training process, I have needed to tweak some of the predefined system configurations to maximize the performance and reduce any kind of lagging and overflow. Adapting the GPU capacity properties to maximize the overall functioning. If I were to work with better equipment, I would have liked to go for longer and deeper trainings, however, I am happy with the performance I obtained when tweaking the parameters I will have discussed in the corresponding section and other strategies such as parallelization.
- Not all limitations have been hardware related, as the official NVIDIA documentation websites were usually corrupted depending on the version. Leading to consulting external pages to solve the mismatching dependencies or installation processes for the frameworks used.

- After consulting the official documentation of the *IsaacLab* repository, I found that even the default reinforcement learning training examples and demonstrations did not work as intended independently of the version. I was not the only one with the same issues as other peoples raised the same issues in different forums, so I inspired in the solutions given by the users with the same problems.

6.3 FUTURE WORK

BIBLIOGRAPHY

- [1] Pietro Bolcato. Understanding vision-language models (vlms): A practical guide, 2024.
- [2] IBM. What are vision-language models? <https://www.ibm.com/think/topics/vision-language-models>, 2024.

7. APPENDIX

7.1 INSTALATION GUIDE

IsaacSim/Lab

7.2 RELEVANT CODE

7.3 PROMPTS USED

Prompts used for the VLM

7.4 RESULT TABLE

7.5 TENSORBOARD ANALYSIS